

Expense Tracker

A command-line program for tracking daily spending, setting budgets, and seeing where the money goes.

Student details

Name	Arth Kashyap
Admission No.	26BCE10526
Programme	B.Tech Computer Science
Institution	VIT (BHOPAL)
Submitted as	VITyarthi Flipped Course Project
Date	September 30, 2026

Project details

Project title	Expense Tracker
Language	Python 3 (standard library only)
Database	SQLite (expenses.db)
Interface	Text menu in the terminal (No GUI)
Source file	expense_tracker.py (about 475 lines)
Git repository	https://github.com/bread1-lang/expense_tracker
Description	A simple command-line program to Track daily expenses, monthly budgets, get warnings before overspending, view a expenses summary, and export everything to CSV. First python programming project

1. Overview

Expense Tracker is a small program that runs in the terminal and helps you keep a record of what you spend. You type in each expense with an amount, a category, a short description and a date. The program saves everything to a file on your computer, so your data is still there the next time you open it.

On top of simple record keeping, you can set a monthly budget for any category. The program warns you when you are close to the limit and tells you clearly when you go over it. At the end of the month you can print a summary that shows the total, the average, and how the spending splits across categories.

The whole project is one Python file and needs nothing to be installed. That was a deliberate choice: anyone with Python can download it and run it straight away.

2. What it can do

Feature	What it does
Add expense	Saves an amount, category, description and date. Pressing Enter uses today's date.
View expenses	Shows the latest 10, all expenses from one month, or everything, in a neat table with a total.
Search	Finds expenses by a word in the description, by category, or both together.
Edit	Change any part of an expense. Pressing Enter keeps the old value.
Delete	Removes an expense after asking for confirmation.
Budgets	Set a monthly limit per category and check the status (OK, WARNING or EXCEEDED).
Monthly summary	Total, count, average, a category breakdown with simple bar graphs, and budget
Categories	status.
CSV export	Nine ready-made categories, and you can add your own.
	Saves all expenses to expenses_export.csv, which opens in Excel or Google Sheets.

3. How it works

The program follows a simple pattern. It starts, makes sure the database exists, and then shows a menu again and again until you choose to exit. Each menu number is connected to one function that does that job.

```
Start
|
v
setup_database()  <- creates tables the first time
|
v
Show main menu  <-----+
|                                     |
v                                     |
Run the chosen function -----+
|
v (choice 0)
Close database, say Goodbye
```

How the code is organised

The file is split into clearly labelled sections, so it is easy to find things:

- **Database setup** - creates the tables and fills in the default categories.
- **Input helpers** - small functions that keep asking until the user types something valid.
- **Showing expenses** - printing tables and fetching rows from the database.
- **Expense options** - add, view, search, edit and delete.
- **Budgets** - set limits and show how much of each is used.
- **Reports** - the monthly summary.
- **Categories and export** - list or add categories, write the CSV file.
- **Main menu** - the loop that ties it all together.

4. How the data is stored

All data lives in one SQLite file called **expenses.db**, created automatically in the same folder the first time you run the program. SQLite is built into Python, so there is no server or setup. The database has three tables.

Table	Columns	Purpose
expenses	id, amount, category, description, expense_date	One row per expense. The id is assigned automatically.
categories	name	The list of allowed categories.
budgets	category, monthly_limit	One monthly limit per category.

Dates are stored as text in the form YYYY-MM-DD. This keeps sorting simple, and finding all the expenses in a month is just a matter of matching the first seven characters (for example 2026-09).

5. Design choices worth mentioning

Input is checked before it is used

Every time the program asks for something, it checks the answer and asks again if it is not valid. Amounts must be positive numbers (commas are allowed, so 1,500 works). Dates must follow YYYY-MM-DD and cannot be in the future. Text fields cannot be empty. Because of this, a typing mistake never crashes the program or saves bad data.

Safe database queries

All values typed by the user are passed to SQLite as parameters instead of being pasted into the SQL text. This prevents SQL injection and also handles awkward characters such as apostrophes in descriptions.

Friendly budget warnings

When you add an expense to a category that has a budget, the program checks that month's total right away. At 80% of the limit (a setting called WARNING_PERCENT that is easy to change) you get a warning, and past 100% you are told the budget has been exceeded.

Ctrl+C is handled

Pressing Ctrl+C in the middle of an action cancels only that action and returns to the menu. Pressing it at the menu exits cleanly.

6. Example run

Below is real output from the program. A 2000 budget was set for Food, then three expenses were added and the September summary was printed.

```
Saved expense #2: Rs. 1500.00 on Food.
! Warning: 98% of the Food budget is used.

Summary for 2026-09
=====
Total spent : Rs. 2250.00
Expenses    : 3
Average     : Rs. 750.00

By category:
  Food          1950.00   86.7% #####
  Transport      300.00   13.3% ##

Budgets:
Category      Budget      Spent      Left      Status
-----
Food          2000.00    1950.00    50.00    WARNING
```

7. Known limitations

It is a first project, so there are some things it does not do yet. Being honest about them is useful, because they make a natural to-do list.

- **Categories cannot be renamed or deleted**, only added.
- **Budget warnings appear only when adding an expense**, not when editing one or when the budget itself is lowered.
- **The currency is fixed** as Rs. in the code (the CURRENCY setting at the top of the file).
- **One user, one file.** There are no accounts, and the database sits next to the script.
- **Text interface only.** There are no charts other than the # bars in the summary.
- **Automated tests are missing**, so changes have to be checked by hand.
- **The database connection opens as soon as the file is loaded**, which makes the code harder to import into other programs or to test.

8. Summary

Expense Tracker does what it set out to do: it records spending, keeps it organised, and gives helpful feedback about budgets. The code is clearly sectioned, validates its input, and stores data in a real database with no extra software. It is a solid base to keep building on.